



IGNISENSE

THE COMPLETE GUIDE TO **IGNISENSE**

Smart Engine Health Diagnostic. What every feature does, how to use it step by step, how the physics and the diagnosis work, and how to run and present it yourself.

TEAM REVORA

PHYSICS-INFORMED ENGINE DIGITAL TWIN

WRITTEN FOR THE TEAM: LEARN IT, RUN IT, REPLICATE THE DEMO

ALL ENGINE VALUES ARE DEMO CALIBRATION, NOT MANUFACTURER DATA

What is in this guide

Read chapters 1–3 to understand and run the app. Chapter 4 is the feature reference: every screen, with the exact clicks. Chapters 5–6 explain the engineering, so you can defend every number. Chapters 7–8 are for the presentation itself.

1	What IgniSense is	7	The five-minute demo
2	Run it on your computer	8	Replicate it tomorrow: checklist
3	Find your way around	9	Build it yourself: phases and project map
4	Feature guide, with steps	10	Key numbers cheat-sheet
5	How the engine simulation works	11	Limitations to state honestly
6	How the diagnosis works	12	Glossary

HOW TO USE THIS GUIDE

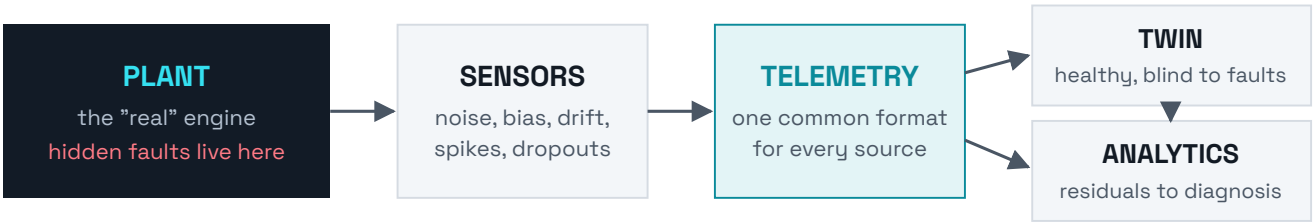
Every "Steps" list is written so you can do it with the app open next to it. Anything marked **SAY** is a sentence you can use when presenting. Words in `this style` are buttons, files or values you will see on screen.

What IgniSense is

A digital twin of a 2.0 L four-cylinder petrol engine that runs entirely in the browser. It shows how a monitoring system can tell that an engine is failing, which part is failing, and how long is left, using only sensor readings.

The idea in one paragraph

Most maintenance is done on a fixed schedule: it either wastes parts or misses failures. Condition monitoring watches the engine instead. IgniSense simulates two engines side by side. The **Plant** is the "real" engine, and it can develop hidden faults such as a wearing oil pump. The **Twin** is a healthy copy of the same physics that only sees the sensor readings and predicts what every sensor *should* read. The **Analytics** compare the two. The gap between what the sensors read and what the healthy Twin expects is the evidence, and everything else (fault name, severity, health score, remaining life) is built from that gap.



THE RULE THAT MAKES IT HONEST

The analytics only ever receive telemetry. The fault state lives in the Plant, in a folder the analytics code is not allowed to import. A lint rule enforces it, so the diagnosis cannot read the answer.

Figure 1. The four parts. Anything that produces telemetry (the simulator, a CSV file, an ESP32, a car's OBD-II port) can feed the same Twin and analytics. All engine values are demo calibration, not real-engine data.

What it can do

CAPABILITY	IN PLAIN WORDS
Live digital twin	A 3D engine, moved by the same equations as the numbers, beside gauges that each show what a healthy engine would read.
Fault diagnosis	Names oil-pump wear, cooling degradation, bearing wear, alternator faults and a misfire in a specific cylinder, with the evidence listed.
Early warning	A statistical detector (CUSUM) notices a small persistent drift before any fixed limit trips.
Sensor-fault detection	Proves from physics that a reading is impossible (for example the coolant "jumping" 25 °C in a tenth of a second) and blames the sensor, not the engine.
Remaining useful life	A straight-line trend with an uncertainty band, shown only when the trend is statistically real.
Calculations	Click any gauge value to see the equation behind it with today's numbers substituted in.
Blind test, real data	A hidden fault case is selected and the monitor must name it. Record and replay CSV, ESP32, OBD-II and phone sensors all use the same analytics.

Run it on your computer

There is no server, no account and no internet needed once it is installed. Everything runs in the browser on your own machine.

What you need

- **Node.js 20.19 or newer**, the LTS version from nodejs.org. Check it by opening a terminal and typing `node --version`.
- A recent **Chrome, Edge or Firefox**. Web Serial (ESP32 / OBD-II) works in Chrome and Edge only.
- **Internet, once**, for the first install. After that it runs offline.

Easiest way: double-click

- 1 Download or copy the project folder to the computer and unzip it.
- 2 On Windows, double-click `start-ignisense.bat`. On macOS or Linux, open a terminal in the folder and run `sh start-ignisense.sh`.
- 3 The first run installs the dependencies. **Keep the window open until it finishes** (a minute or two, needs internet). Later runs skip this and take a few seconds.
- 4 It builds the app and opens `http://localhost:4173` in your browser.
- 5 To stop it, close the window (or press `Ctrl+C`).

From a terminal

COMMAND	WHAT IT DOES
<code>npm install</code>	Once, needs internet. Downloads everything the app needs.
<code>npm run demo</code>	Builds the app and opens it at <code>localhost:4173</code> . Use this on the presentation laptop. It works offline.
<code>npm run dev</code>	Development mode at <code>localhost:5173</code> with live reload. Use it when you are changing code.
<code>npm test</code>	Runs all 363 tests (physics, analytics, scenarios).
<code>npm run lint</code>	Style and architecture checks, including the rule that keeps the analytics blind.

If something goes wrong

PROBLEM	FIX
<code>'tsc' is not recognized</code>	A previous install did not finish. Delete the <code>node_modules</code> folder and run <code>start-ignisense.bat</code> again. Keep the window open until the install ends. (The launcher now repairs this by itself.)
"Node.js is not installed"	Install Node from nodejs.org , then close every window and open a new one . A window opened before the install does not see Node.
"Port 4173 is in use"	IgniSense is probably already running in another window. Close it, or open <code>localhost:4173</code> directly.
3D view is slow	The fps number is in the 3D view's top-right corner. The scene lowers its own effects under 28 fps. As a last resort, add <code>?no3d</code> to the address (<code>localhost:4173/?no3d</code>): everything works, without the 3D engine.

Find your way around

One header, seven pages, and one drawer for controlling the simulation.

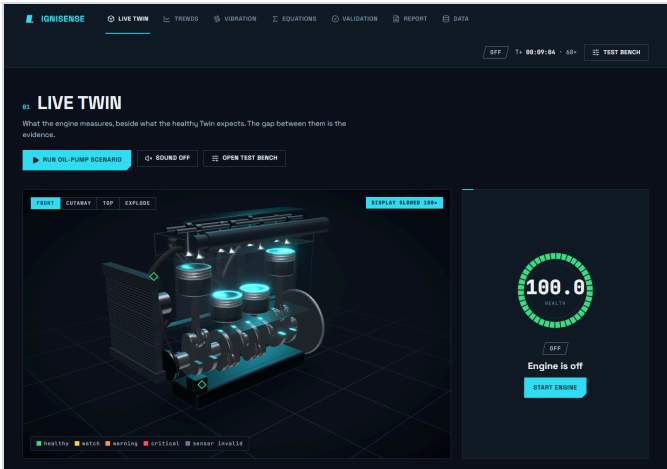


Figure 2. The Live Twin page with the engine off. The header (top) has the seven pages, the engine state, the simulated clock with the time-warp, and the Test bench button.

PAGE	WHAT IT IS FOR
Live Twin	The home view: 3D engine, health, gauges with Twin markers, the diagnosis card, alerts, cause-and-effect graph.
Trends	Measured vs Twin over time, fault and alert markers, and the two statistical detectors (D^2 and CUSUM).
Vibration	The vibration spectrum, the misfire polar plot that points at the failing cylinder, and crank-speed ripple.
Math	Every equation the app uses, with live numbers substituted.
Validation	Hand-calculated values beside the model's, computed live. All should be green.
Report	A one-page maintenance report you can print or save as PDF.
Data	Raw telemetry beside the Twin, plus recording, CSV replay and hardware connections.

The header, left to right

- **Page tabs.** Click to switch. The address changes too (`#trends`, `#math`...), so any page can be linked.
- **Engine state badge.** OFF, STARTING, WARMUP, RUNNING (STEADY or TRANSIENT), or SHUTDOWN.
- **Clock.** `T+ hh:mm:ss · 60×` is *simulated* time and the time-warp. At 60×, one real second is one simulated minute, so a half-hour warm-up takes about half a minute.
- **Source badge.** Appears as `REPLAY` or `LIVE` whenever something other than the simulator feeds the monitor.
- **Test bench.** Opens the control drawer (next chapter).

COLOURS MEAN SOMETHING

Green = healthy, yellow = watch, orange = warning, red = critical, grey = sensor invalid. These are reserved for engine and sensor *state*. Cyan is the brand and interactive colour. The **white** tick or pointer is always the Twin's expected value.

Feature guide, with steps

Each feature has the same layout: what it is, the steps to use it, what to look for, and (where useful) how it works underneath.

4.1 The Test bench (control everything from here)

Open it with `Test bench` in the header or `Open test bench` on the Live Twin page. It slides in from the left. **Buttons that do something** (Inject fault, Repair, Start/Stop engine, Run/Stop scenario) close the drawer so you can watch the result. **Settings** (sliders, warp, fan) leave it open. `Esc` or `Close X` also closes it.

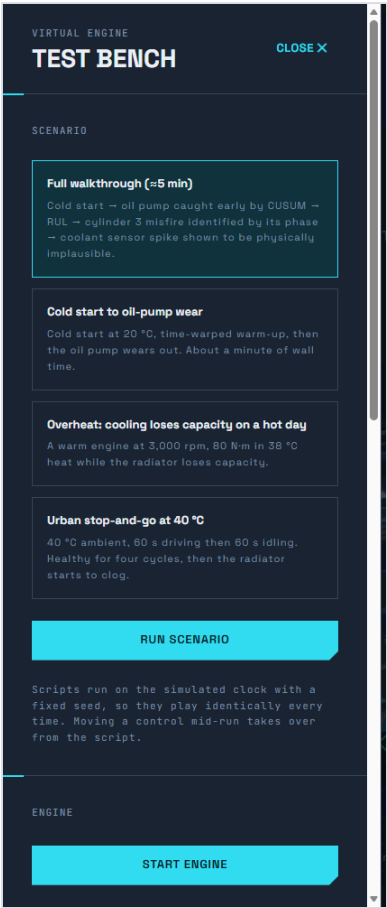


Figure 3. The Test bench drawer. Top to bottom: Scenario, Engine, Blind test, Fault injection.

SECTION / CONTROL	WHAT IT DOES
Scenario list + <code>Run scenario</code>	Plays a scripted story on the simulated clock with a fixed seed, so it is identical every time. Four scenarios: Full walkthrough, Cold start to oil-pump wear, Overheat, Urban stop-and-go (see 4.3).
<code>Start engine</code> / <code>Stop engine</code>	Cranks the engine (STARTING, then WARMUP, then RUNNING) or shuts it down.
Target RPM	800 (idle) to 7,000. Redline is at 6,000. Default 3,000.
Load (N·m)	Brake torque applied to the engine, 0 to peak torque (190). Default 80. The app converts it to a load fraction $L = \text{torque} \div \text{the most the engine can make at that speed}$.
Ambient (°C)	Outside air temperature, -10 to 50. Default 30.
Radiator fan	AUTO (switches on above 98 °C with hysteresis), ON, or OFF.

Time-warp	1×, 10× or 60×. Use 60× to skip warm-up, 10× to watch a fault develop, 1× to talk through something.
Start blind test	See 4.8.
Fault / Severity / Onset	See 4.2.
Inject fault / Repair	Applies the chosen fault, or removes all faults.

4.2 Injecting a fault

A fault is applied to the hidden Plant. Nothing on screen tells the monitor which one you chose: it has to find it in the sensors.

- 1 Start the engine () and let it warm up. Set time-warp to to speed this up. Wait until the badge says · .
- 2 Open the Test bench. Under **Fault injection**, choose a **Fault** from the list.
- 3 Set the **Severity** (0 to 1). Higher is worse. The small note underneath tells you the resulting health factor (for example severity 0.60 leaves oil-pump health at 0.55).
- 4 Choose the **Onset**: ramps in over 120 simulated seconds, is immediate, keeps the fault growing (faster under load, heat and speed) so the remaining-life estimate has a trend to follow.
- 5 Click . The drawer closes. Watch the Live Twin page.
- 6 To clear it, open the bench and click . Alerts clear after their hold time (about 20 simulated seconds per level).

FAULT	WHAT IT DOES TO THE ENGINE	WHAT THE MONITOR SEES
Oil-pump wear	Pump health = $1 - 0.75 \times \text{severity}$. Oil pressure drops, even at idle.	Pressure far below the Twin's expectation; lubrication subsystem falls; CUSUM warns early.
Cooling degradation	Radiator loses capacity (health = $1 - 0.8 \times \text{severity}$). Coolant runs hot under load.	Coolant residual and its rate of change; thermal subsystem falls.
Cylinder misfire	Choose C1 – C4 . That cylinder's combustion health = $1 - \text{severity}$ (1 = dead cylinder).	Half-order line in the crank-speed spectrum; the polar plot names the cylinder.
Bearing wear	Lowers oil pressure and loosens the crank: more 1× vibration and knock.	Low pressure plus a 1× rise plus impacts is named "bearing wear" inside the lubrication diagnosis.
Alternator fault	Charging voltage sags.	Bus-voltage residual; electrical subsystem falls.
Sensor fault	Leaves the engine healthy and breaks one <i>sensor</i> : choose COOLANT, OIL T, PRESS or VOLT, and SPIKE, STUCK, DRIFT or DEAD.	The monitor proves the reading is impossible and reports a sensor fault, not an engine fault.

4.3 Scenarios (one-button stories)

SCENARIO	WHAT HAPPENS
Full walkthrough (≈ 5 min)	The whole demo on one button: cold start, oil pump caught early by CUSUM, remaining life, cylinder 3 misfire named by its phase, then a coolant sensor spike proven impossible. Each step waits for what the monitor concluded before moving on.

Cold start to oil-pump wear	Cold start at 20 °C, time-warped warm-up, then the oil pump wears out. About a minute of real time.
Overheat	A warm engine at 3,000 rpm and 80 N·m in 38 °C heat while the radiator loses capacity.
Hot city	Stop-and-go at 40 °C ambient: 60 s of driving, 60 s of idling, four cycles, then the radiator starts to clog.

- 1 Live Twin: click **Run oil-pump scenario** for the quickest story. Or open the Test bench, pick a scenario card, click **Run scenario**.
- 2 A narration strip appears under the page title: step dots, a title and a caption for what is happening.
- 3 Use **Pause** on the strip to talk over it, **Resume** to continue, **Restart** to replay, **Stop** to end.
- 4 Moving any control mid-run takes over from the script.

WHY IT IS IDENTICAL EVERY TIME

All randomness goes through one seeded random-number generator, and scenarios run on the *simulated* clock, not the wall clock. A tested guarantee: three runs at different speeds give the same telemetry, step times and alerts.

4.4 Live Twin page

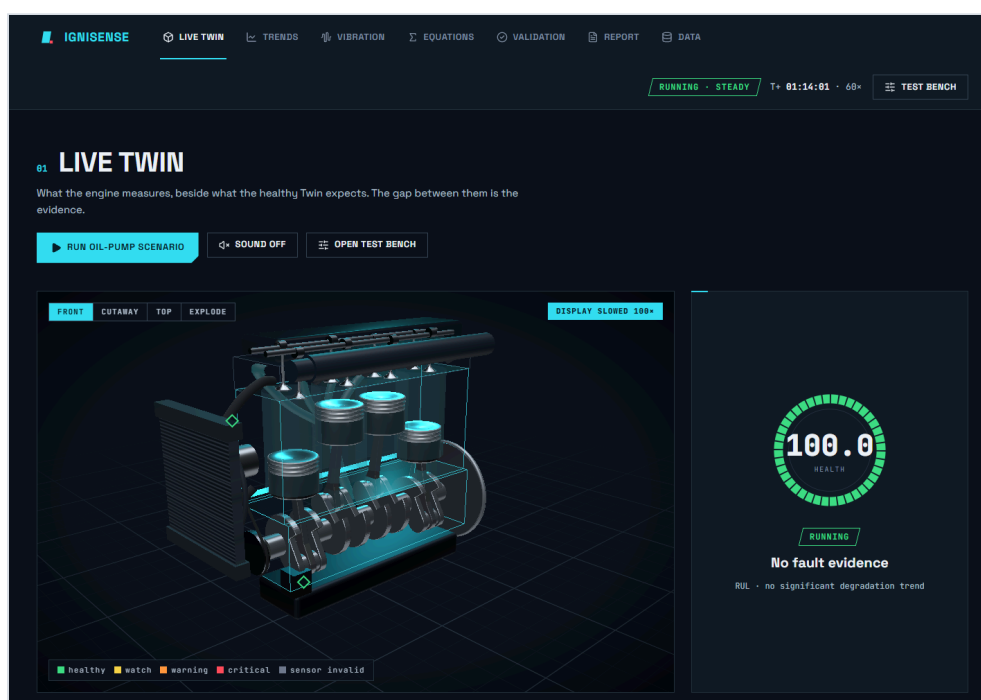


Figure 4. The engine running at about 3,000 rpm. The 3D engine on the left, the health ring on the right, subsystem health underneath.

THE 3D ENGINE

- Built from code, not a downloaded model. The piston positions come from the slider-crank equation, so TDC and BDC line up with crank angle 0° and 180°.
- It is slowed 100× so you can see the motion. The label **DISPLAY SLOWED 100×** and the readout **engine 3,000 rpm · shown at 30 rpm** say so honestly.
- Cylinders flash in firing order 1-3-4-2. A cylinder that the monitor has diagnosed as misfiring stops flashing. The block shakes at the real second-order vibration.

- The block turns from dark to cyan as the coolant warms. The oil gallery glows with pressure. A faulty subsystem's parts pulse in the status colour.

Steps:

- 1 Click **FRONT**, **CUTAWAY**, **TOP** or **EXPLODE** (top-left of the view) for a camera preset. Drag to orbit, scroll to zoom.
- 2 Click a diamond marker (oil pump, radiator) or hover it: a label slides out with that part's health.
- 3 Click the label to open a side panel with that part's sensors: measured, expected and residual. **View calculation** opens the equation for it.
- 4 Read the frame rate at the top of the view (**60 fps · 71 meshes**). **reduced effects** means it lowered its own quality to stay smooth.

THE HEALTH RING AND SUBSYSTEMS

The ring shows overall health from 0 to 100, coloured by the overall alert level (not by score bands). Under the number: the engine state or the alert class, and **"limited by ..."** naming the subsystem that drives a **WARNING** or **CRITICAL** level (the overall number is a weighted average, so it can look mild beside one failing subsystem), the name of the diagnosed fault ("No fault evidence" when healthy), and a **remaining-life** line when a significant trend exists (otherwise it says what the statistics layer is doing, for example "learning the healthy baseline").

Below are six subsystems, each 0 to 100, with their weights: Lubrication 0.25, Thermal 0.25, Mechanical vibration 0.20, Combustion 0.15, Electrical 0.10, Sensor integrity 0.05. A dash means it is not monitored while the engine is off.

SIGNALS: THE GAUGES



Figure 5. Signals. The tachometer and six bar gauges. The white tick is the Twin's expected value.

- **Engine speed** is a car-style tachometer (0 to 7 in thousands of rpm, redline from 6,000). The white pointer on the rim is the speed the Twin expects.
- **Engine load, Coolant, Oil temp, Oil pressure, Voltage, Vibration RMS** are bar gauges. The white tick and **TWIN** value show what a healthy engine would read *right now*, at the same speed, load and ambient.
- **Δ** is the residual (measured – expected). It turns coloured once it leaves the normal noise band (more than 2 σ).
- The coloured strip under a bar marks warning and critical zones. The small square in the corner is the sensor quality: green valid, amber degraded, red failed, grey unavailable.
- A dead sensor shows **--** with a hatched track. It is never shown as 0.

Steps: click any gauge's number, or its **$f(x)$** mark, to open its calculation (4.7).

DIAGNOSIS CARD AND ALERTS



Figure 6. During an oil-pump fault: the explanation card (left) and the alert list, newest first (right).

- **Probable fault** with a severity badge and an **evidence score**. It is deliberately called a "diagnostic evidence score", never "confidence %". Classes: under 0.30 weak, 0.30–0.55 possible, 0.55–0.75 probable, over 0.75 strong.
- **Why the system thinks this** is a checklist of named symptoms, each with its status colour.
- **Recommended action**, then **Model status** (physics rules, anomaly model, remaining life) and **Confidence in this output** (evidence strength, data quality, model agreement, uncertainty).
- **Alerts** are INFO, WATCH, WARNING or CRITICAL. Each carries the time, the subsystem, measured vs expected, the residual, how long it has persisted and whether a rule or a statistic raised it.

CAUSE AND EFFECT GRAPH

At the bottom of the page a graph links each fault the monitor can name to its physical effect and to the sensor that sees it. Nodes light up from the live symptom risks, so you can trace a reading back to its cause while the evidence builds.

84 CAUSE AND EFFECT

Each fault the monitor can name, the physical effect it has, and the sensor that sees it (draft 96). Nodes light up from the live symptom risks, so the path from a reading back to its cause is visible while the evidence builds.



Figure 7. Cause and effect graph.

4.5 Trends

Charts of measured against expected over *simulated* time (they scroll as the clock runs).

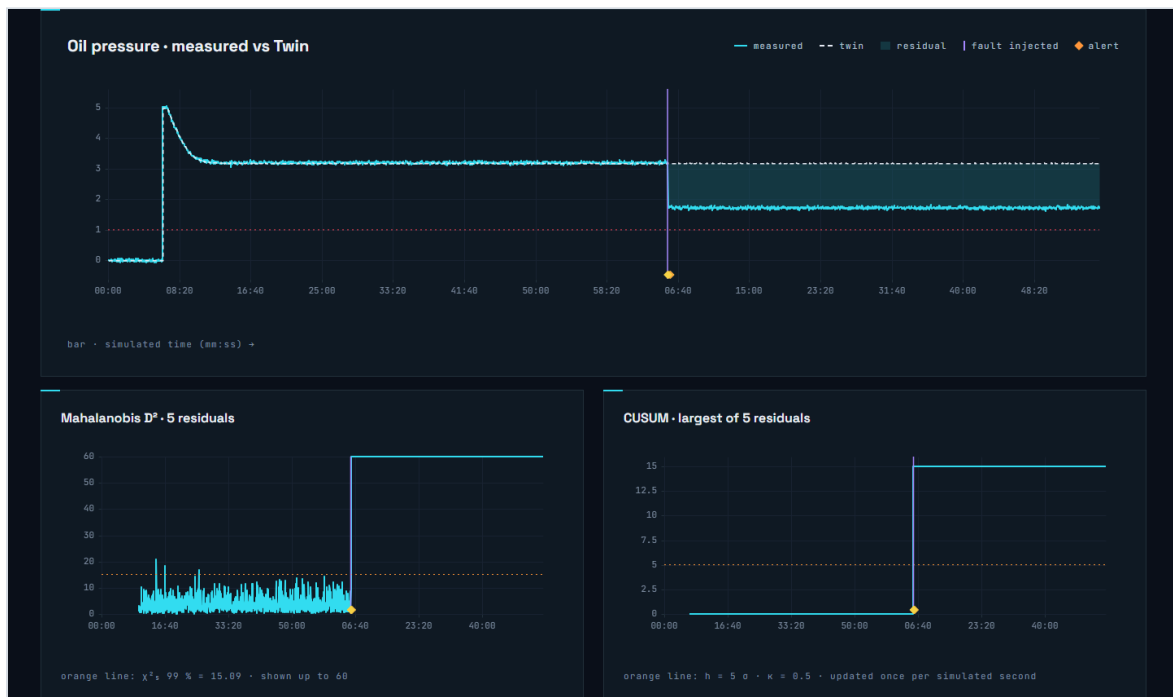


Figure 8. Trends after an oil-pump fault: the measured line drops away from the Twin, violet lines mark the injection, diamonds mark the alerts.

- **Charts:** oil pressure, coolant, oil temperature, bus voltage, and overall health. The solid line is measured, the dashed line is the Twin, the shaded band is the residual.
- **Markers:** violet vertical lines are fault injections; diamonds are alerts.
- **Statistics:** D^2 (Mahalanobis distance) flags an unusual *combination* of residuals against its 99 % limit (15.09). **CUSUM** accumulates small persistent shifts and crosses its limit of 5 well before a fixed threshold trips.

Steps: inject a fault (4.2) and open Trends. Watch CUSUM on oil pressure cross 5 *before* the rule-based alert appears: that is the early warning.

4.6 Vibration and misfire

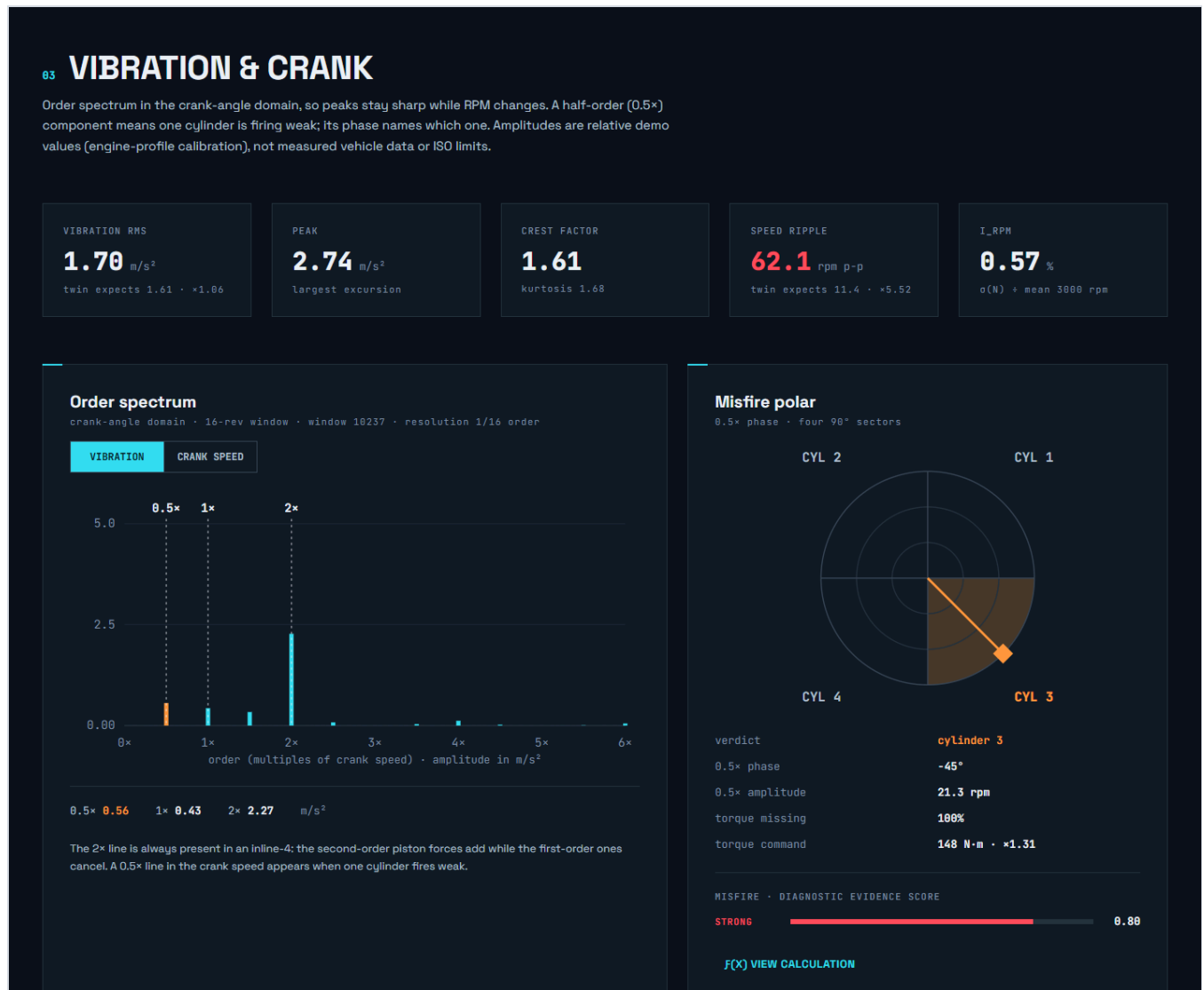


Figure 9. Vibration page during a cylinder 3 misfire: the half-order line, the polar plot and the crank-speed ripple.

- **Metrics:** vibration RMS, peak, crest factor, kurtosis, and the crank-speed irregularity.
- **Order spectrum:** vibration strength against *engine order* (multiples of crank speed), with cursors at 0.5x, 1x and 2x. A four-cylinder engine shakes at 2x. A misfire adds a 0.5x line.
- **Misfire polar plot:** a dot at the phase of the 0.5x component. It falls in the sector of the misfiring cylinder: C1 +45°, C2 +135°, C3 -45°, C4 -135°. The size of the dot tells how much torque is missing.
- **Waveforms:** the accelerometer signal and the crank-speed ripple over 16 revolutions.
- **Device accelerometer:** a panel that runs a phone's motion sensor through the same FFT (needs a phone, https or localhost).

Steps:

- 1 Run the engine at about 3,000 rpm. Open Vibration and note the clean 2x line.
- 2 Test bench, Fault: **Cylinder misfire**, Cylinder: **C3**, click **Inject fault**.

- 3 Wait a few seconds. A 0.5× line appears, the polar dot jumps to the C3 sector (−45°), and the diagnosis card names **Cylinder 3 misfire**.
- 4 Click **Sound on** (Live Twin): one exhaust pulse in four goes missing, and the engine audibly stumbles.

4.7 Calculations

The app can show its own working. Every equation is registered with its symbols, units and a template, so the calculation view can substitute today's live numbers into it.

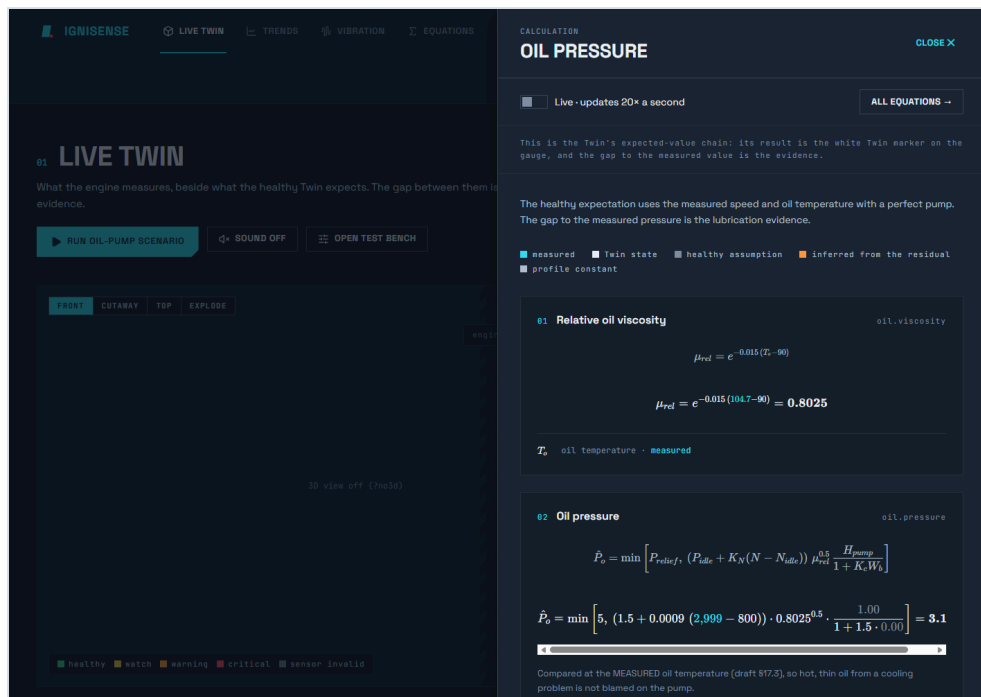


Figure 10. The math drawer for oil pressure: the chain of equations with today's numbers, and the check that the result equals the gauge.

- 1 On Live Twin, click a gauge's number (for example oil pressure). A drawer opens from the right.
- 2 Read the **chain of equations**. Numbers are colour-tagged: measured, Twin state, healthy assumption, inferred, or constant.
- 3 Look at the **checks against the screen** table at the bottom. Each row shows the equation's result beside the gauge value and a **EQUAL**.
- 4 Flip the switch at the top (**Live · updates 20× a second**) to **Holding these numbers**. It freezes the numbers so you can read them out while the engine runs.
- 5 For the full list, open the **Math** page: pick a gauge to see its live chain, and scroll for all 28 registered equations with their symbols and units.

64 EQUATIONS

Every value on screen comes from a registered equation. Select a gauge to see its calculation with current values substituted: cyan is measured, white is the Twin's own state, grey is a healthy-engine assumption, orange is inferred by the monitor. The chain shown is the Twin's expected-value chain; its result is the white Twin marker on the gauge.

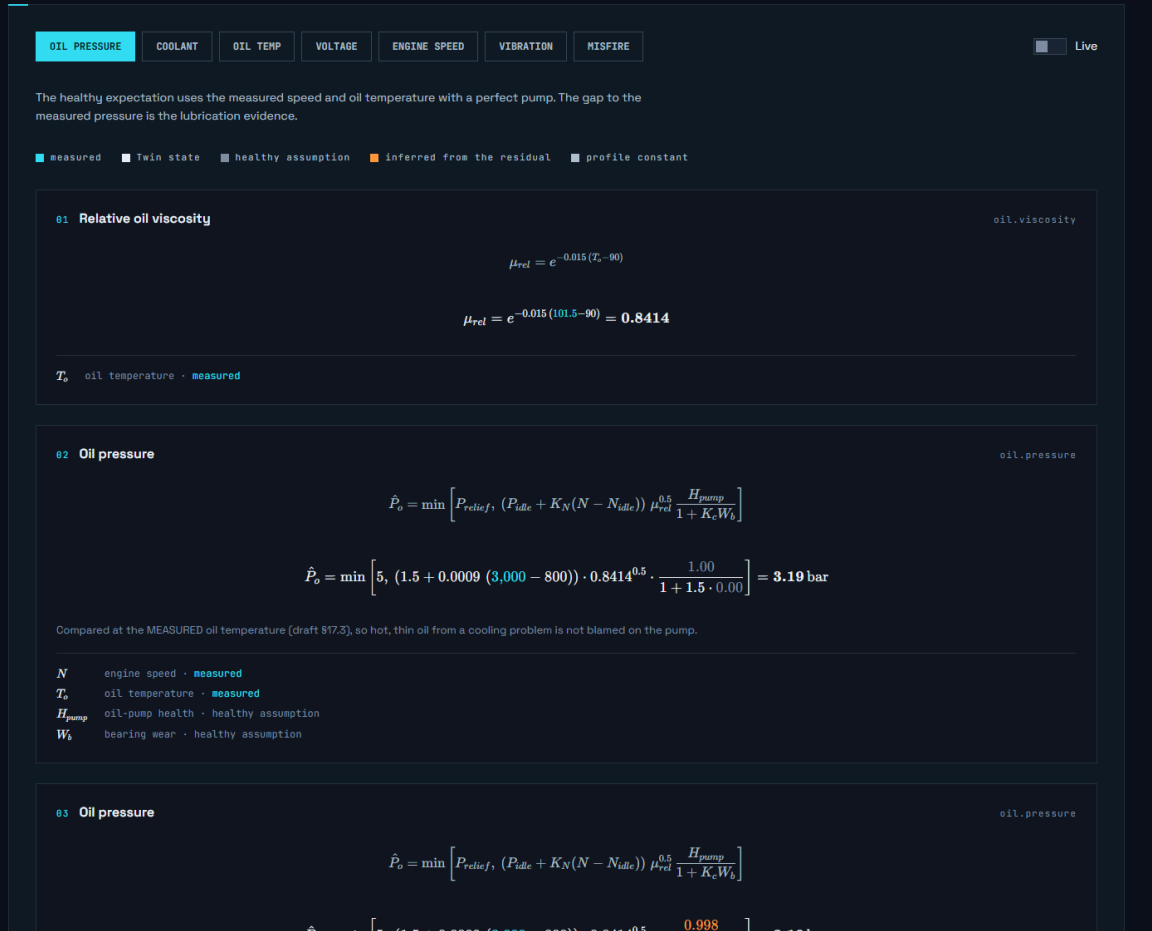


Figure 11. The Math page.

SAY

Every number on this screen has an equation behind it, and you can click it. We test that the numbers substituted into the equation equal the gauge on the screen, in six different engine states.

4.8 Blind test

Proof that the monitor is not reading the injected fault. A hidden fault is chosen inside the simulation worker; the page only ever learns the card's position, never its content.

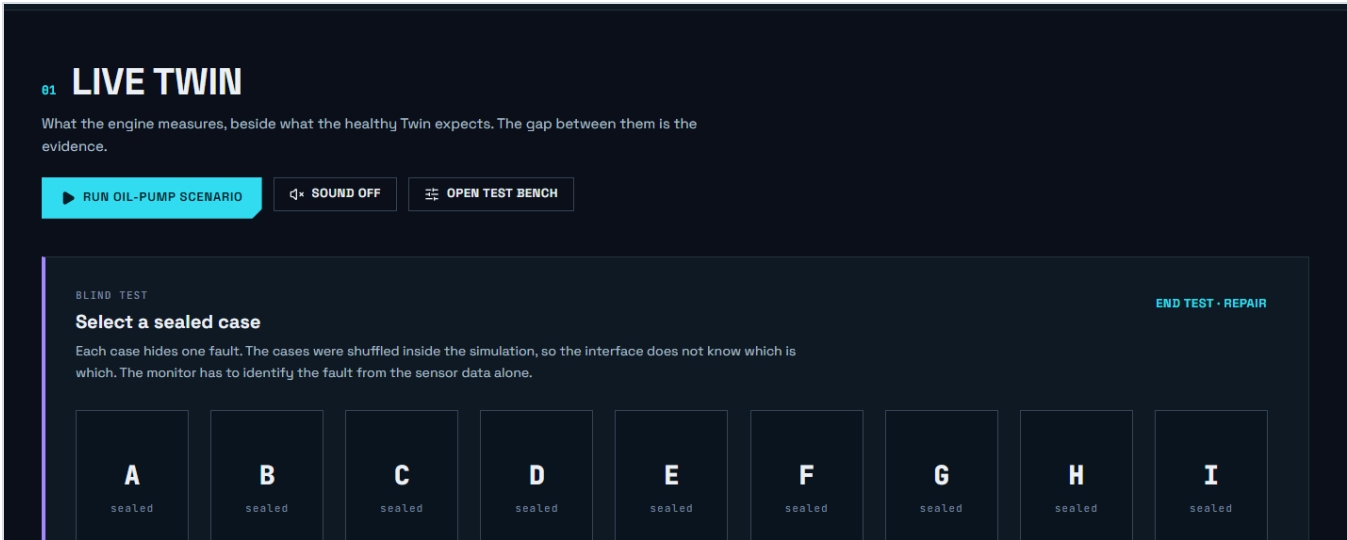


Figure 12. The blind test: nine sealed cases, shuffled by the worker.

- 1 Open the Test bench and click **Start blind test**. The drawer closes and the Live Twin page shows nine sealed cases.
- 2 Ask a visitor to select a case. Nobody, including you, knows what is on it.
- 3 The engine runs at part load and 10× warp. Watch the diagnosis card: within seconds to about a minute the monitor names a fault.
- 4 Press **Reveal the card**. The strip shows what was on the card and whether the monitor was **CORRECT**.
- 5 Click **New blind test** for another round.

The nine cards are: oil-pump wear, cooling degradation, a misfire in each of the four cylinders, worn crank bearings, an alternator losing output, and a stuck coolant sensor. Every card is named correctly in the tests.

4.9 Validation

The screenshot shows the 'IGNISENSE VALIDATION' interface. It has a header '05 IGNISENSE VALIDATION' and a sub-header 'Hand-calculated and table values beside the model's, computed live from the same physics and analytics the simulator runs. Rows pass within 2 %, the same rule as the test suite.' In the top right, it says '26/26 passing · 0 pending'. Below is a table with 4 columns: CHECK, EXPECTED, MODEL, and STATUS. The table contains 26 rows of data, all with a 'PASS' status.

CHECK	EXPECTED	MODEL	STATUS
ω at 3,000 rpm	314.16 rad/s	314.16	PASS
Fuel power · 3,000 rpm / 80 Nm	118.8 kW	118.8	PASS
Coolant heat · 3,000 rpm / 80 Nm	31.9 kW	31.9	PASS
Steady coolant · 3,000 rpm / 80 Nm	93.8 °C	93.8	PASS
Oil pressure · cold start 20 °C	2.54 bar	2.54	PASS
Oil pressure · pump 0.4, hot idle	0.56 bar	0.56	PASS
Max coolant heating rate	0.815 K/s	0.815	PASS
F _z peak at 800 rpm	179 N	179	PASS
F _z peak at 5,000 rpm	2,517 N	2,517	PASS
F _z peak at 8,000 rpm	10,868 N	10,868	PASS
Healthy crank-speed ripple · 3,000 rpm	11.0 rpm p-p	11.1	PASS
Cyl 1 misfire · speed ripple	61.0 rpm p-p	61.2	PASS
Cyl 1 misfire · 0.5x amplitude	21.0 rpm	21.2	PASS
Cyl 1 misfire · 0.5x phase	45 °	45	PASS

Figure 13. Validation: 26 live checks, each a hand-calculated value against the model.

The Validation page computes the physics live in your browser and compares each result with a value worked out by hand. Examples: angular speed 314.16 rad/s, $F_2 = 2,517 \text{ N}$ at 3,000 rpm, cold-start oil pressure 2.54 bar, the 4/3 misfire torque ratio, the χ^2 limit 15.09, and the health example 69.4. All rows should read PASS. It is your answer to "how do you know the model is right?"

4.10 Maintenance report

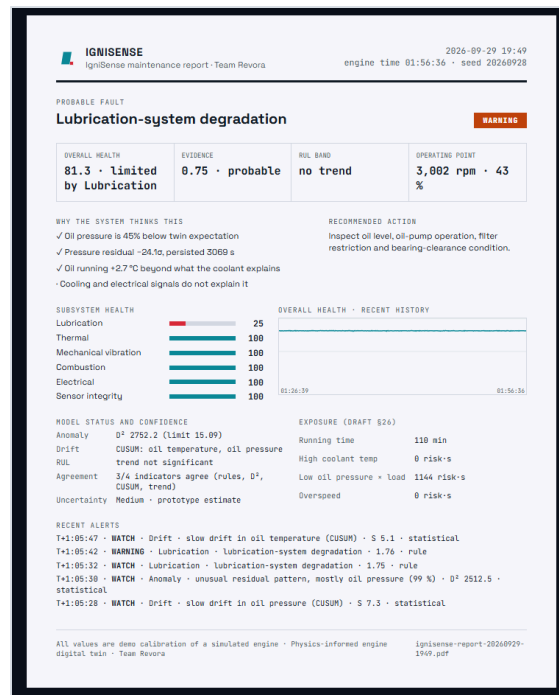


Figure 14. The report: a frozen snapshot of the live monitor.

- 1 Run a fault so there is something to report.
- 2 Open **Report**. It shows "*IgniSense maintenance report · Team Revora*": the verdict, the evidence, the recommended action, the remaining-life band, subsystem bars, health history, model status and confidence, exposure counters and recent alerts.
- 3 Click **Refresh from engine** to take a new snapshot (it does not update by itself).
- 4 Click **Print / save PDF**. Choose "Save as PDF" as the printer. It prints on exactly one A4 page and the file is named `ignisense-report-<time>`.

4.11 Engine sound

The sound is built from what the monitor *measures*, not from the hidden fault. Each cylinder fires an exhaust pressure pulse as loud as its measured power stroke, in the order 1-3-4-2, at the **real** engine speed (the 3D view is slowed, the sound is not). The pulses pass through a small model of an exhaust system (a pipe resonance, a muffler boost, and saturation that gets rougher under load).

- 1 On Live Twin click **Sound off** (it becomes **Sound on**). The click is required by the browser.
- 2 Start the engine. Change the target RPM and load: pitch follows speed smoothly, and it gets louder and brighter under load.
- 3 Inject a cylinder misfire: one pulse in four disappears and the engine stumbles.

4.12 Data & sources: recording, replay and real hardware

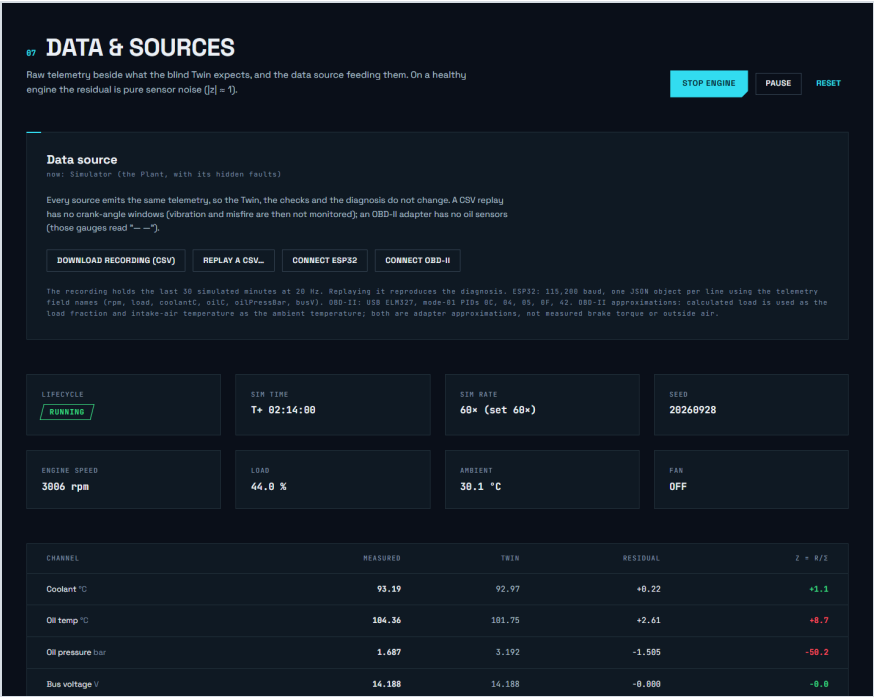


Figure 15. Data & sources: raw telemetry beside the Twin's expectation, and the data-source controls.

BUTTON	WHAT IT DOES
Download recording	Saves the last 30 simulated minutes as a CSV file (the same telemetry the analytics see).
Replay a CSV	Loads a recording. The header shows REPLAY . The same Twin and analytics process it, and replaying a recorded pump-fault run reproduces the diagnosis. It stops at the end of the file.
Connect ESP32	Web Serial over USB, JSON lines from a microcontroller with real sensors. Chrome or Edge only.
Connect OBD-II	A USB ELM327 adapter on a real car: engine speed, load, coolant, intake temperature, module voltage. The oil channels are simply not fitted, and that raises no false alarm.
Back to simulator	Returns to the built-in Plant.
Pause / Reset	Pause freezes the simulated clock; Reset restarts the simulation from a cold engine with the same seed.

NOT YET TESTED

The replay path is tested end to end. The ESP32 and OBD-II paths are tested against parsers and simulated streams, but not yet against real hardware. Say so if asked.

How the engine simulation works

Two clocks run inside a Web Worker: a slow physics loop for temperatures and pressure, and a fast crank-angle model for vibration and misfire.

The engine and its numbers

PARAMETER	VALUE
Bore × stroke	86 × 86 mm, crank radius $r = 43$ mm
Con-rod length l ; ratio $\lambda = r/l$	145 mm; 0.297
Displacement	1,998 cc
Firing order; crank throws	1-3-4-2; 0°, 180°, 180°, 0°
Idle / redline	800 / 6,000 rpm
Peak torque curve	$T_{\max}(N) = 190 [1 - 0.53 ((N - 4000)/4000)^2]$ N·m
Load	$L = T_{\text{brake}} / T_{\max}(N)$

The slow loop (20 steps per simulated second)

- **Friction and energy:** friction pressure FMEP = $(97 + 15n + 5n^2) \cdot (1 + \max(0, 90 - T_{\text{oil}})/70)$ kPa with $n = \text{rpm}/1000$. Cold oil is thick, so friction is higher when cold. Indicated efficiency is $\eta = 0.38 (0.70 + 0.30 L)$. Fuel power = brake power ÷ efficiency.
- **Coolant temperature:** $C_{\text{th}} dT/dt = (\text{heat into coolant}) - UA \cdot (T - T_{\text{ambient}})$. Heat into coolant is $(0.34 - 0.12 L)$ of fuel power. The radiator UA depends on a thermostat that opens linearly from 82 to 95 °C, and on the fan, which switches on above 98 °C with hysteresis. $C_{\text{th}} = 100$ kJ/K.
- **Oil temperature:** a second heat balance, warmed by friction and by the coolant, cooled by the air.
- **Oil pressure:** $\min[P_{\text{relief}}, (P_{\text{idle}} + K_N (N - N_{\text{idle}})) \cdot \mu_{\text{rel}}^{0.5} \cdot H_{\text{pump}} / (1 + K_c W_b)]$ with viscosity $\mu_{\text{rel}} = e^{(-0.015 (T_{\text{oil}} - 90))}$. Cold and thick gives 2.54 bar at idle; hot idle gives 1.39 bar.
- **Battery voltage:** $V = V_{\text{bat}} + H_{\text{alt}} \cdot g(N) \cdot (V_{\text{reg}} - V_{\text{bat}}) - \Delta V_{\text{load}}$, with $g(N) = 1 - e^{(-N/600)}$.

HEALTH FACTORS

A fault is a number between 0 and 1 that multiplies part of the physics (pump health, cooling health, cylinder combustion health). Every health factor is 1 in a healthy engine. The **Twin** runs the *same* functions with every health factor forced to 1, driven only by the *measured* speed, load and ambient temperature.

The sensor model

```
reading = true value + bias + drift(t) + noise
plus: dropout (reads null, never 0 or NaN), stuck, spike
```

Noise levels (one standard deviation): rpm 5, load 0.004, ambient 0.1 °C, coolant 0.2 °C, oil temperature 0.3 °C, oil pressure 0.03 bar, bus voltage 0.03 V.

The fast loop: one crank angle drives everything

- One 720° cycle is integrated in 0.5° steps. Each cylinder makes a **half-sine torque pulse** during its power stroke; the four pulses are offset by 180° in the order 1-3-4-2.

- The crank speed changes as torque in exceeds the load (energy form: the kinetic energy changes by the area between the torque and the load). A governor holds the mean speed.
- That one model produces the **crank-speed ripple** (about 11 rpm peak to peak, healthy, at 3,000 rpm / 80 N·m), the **vibration signal** (second-order force $F_2 = 4 m r \omega^2 \lambda \cos 2\theta$ plus combustion orders, 1× imbalance, knock impacts and noise), and what the 3D pistons do (slider-crank equation).
- A window of 16 revolutions is sampled at 512 points per revolution (8,192 points), which is what the FFT analyses. Sampling per revolution, not per second, is called **order tracking**: a fault at "0.5×" stays at 0.5× at any speed.

Why the same run is always identical

Every random number comes from one seeded generator (`src/lib/rng.ts`). The scenarios run on the simulated clock. So the oil-pump and walkthrough scenarios play the same way on every computer.

How the diagnosis works

From raw readings to a named fault, in eight small steps. Each is simple enough to explain on a whiteboard.

- 1 Residual.** $r = \text{measured} - \text{expected}$. Normalised: $z = r / \sigma$, where σ is that sensor's noise. $|z|$ under 2 is noise; well above 2 is a real gap.
- 2 Sensor sanity first.** Before any reading is trusted it is checked: is the rate of change physically possible (coolant cannot heat faster than **0.81 K/s**), is it stuck, has it dropped out, does the oil temperature follow the coolant? A reading that fails is left out of the diagnosis and reported as a sensor fault.
- 3 Risk.** Each residual becomes a 0-to-1 risk with a warning and a critical value: $R = \text{clip}((x - W)/(C - W), 0, 1)$.
- 4 Fault evidence score.** For each fault, $S = \Sigma(\text{weight} \times \text{risk}) / \Sigma(\text{weight})$. Weights, for example lubrication: 0.35 low oil pressure, 0.20 pressure residual, 0.20 oil temperature, 0.15 vibration, 0.10 pressure decay. The highest score names the fault.
- 5 Health.** Each subsystem gets 0-100; overall $H = 100 (1 - \Sigma w R / \Sigma w)$. Critical overrides: oil-pressure risk above 0.95 goes straight to CRITICAL.
- 6 Alert hysteresis.** To avoid flicker: NORMAL to WATCH needs risk above 0.30 for 5 s; WATCH to WARNING above 0.55 for 10 s; WARNING to CRITICAL above 0.85 for 5 s; WARNING clears only below 0.40 for 20 s (all in simulated seconds).
- 7 Statistics.** CUSUM: $S = \max(0, S + z - 0.5)$, alarm at 5. A residual that sits at $z = 1$ never crosses a 3σ limit, yet CUSUM passes 5 in about 11 samples. D^2 : the distance of all five residuals together from the healthy baseline learned in the first 60 s, against the χ^2 99 % limit of 15.09.
- 8 Remaining life.** Fit a straight line to the recent health of the worst subsystem. $RUL = (H - H_{\text{fail}}) / |\text{slope}|$, with a band from the slope's standard error ($\pm 2 s_b$). It is shown **only** if the slope is negative and significant, and it is in simulated time at the present load.

Naming the misfiring cylinder

A misfire removes one power pulse every two revolutions. That repeats at **half the crank frequency** (order 0.5). The cylinders fire 180° of crank apart, which is 90° of phase at half order, so each cylinder has its own phase:

CYLINDER	FIRES AT	0.5× PHASE
1	0°	+45°
3	180°	-45°
4	360°	-135°
2	540°	+135°

The governor keeps the mean speed, so with one cylinder out the other three must do four cylinders' work: the torque command rises by exactly **4/3** (in the app, about 111 to 148 N·m).

Proving a sensor lied

```
fastest possible heating = (max heat into coolant) / C_th
                        = 81,500 W / 100,000 J/K ≈ 0.8 K/s
a reading that jumps 25 °C in 0.10 s implies about 250 K/s, over 300 times the limit
```

The engine cannot do that, so the sensor failed. The other sensors and the Twin agree with each other, so the cooling system is **not** blamed.

WORDING RULES THE APP FOLLOWS

A weighted fault score is a "diagnostic evidence score", never a confidence percentage. Vibration thresholds are "engine-profile calibration values", never ISO limits. A crank journal-bearing fault is never labelled with rolling-bearing frequencies (BPFO/BPFI), because those formulas do not apply.

The five-minute demo

Built around three moments people remember: a fault they choose themselves, named from the sensors alone; a misfire that names its cylinder; and a sensor fault the system refuses to believe.

BEFORE YOU START

Run `start-ignisense.bat` (or `npm run demo`). Check the fps number in the 3D view: about 60 is right, under about 20 means reload with `?no3d`. Volume up (sound starts only after you click `Sound on`). Two ways to run it: **Safe** is Test bench → Scenario → **Full walkthrough** → `Run scenario`, which drives every step for you (use `Pause` to talk). **Live** is doing the clicks yourself, best when a visitor picks the fault.

TIME	MOMENT	DO THIS / POINT AT THIS
0:00–0:40	The problem	<code>SAY</code> Fixed-interval maintenance wastes parts or misses failures. IgniSense has two engines: the "real" one with hidden faults, and a healthy twin that only sees sensor readings. The gap between them is the evidence.
0:40–1:20	Cold start	Start at 20 °C, idle. Oil pressure reads about 2.5 bar (cold oil is thick). The white Twin marker sits on each needle. Pistons are slowed 100×. At 60× the half-hour warm-up passes in about half a minute. Click the oil-pressure value: the drawer shows the equation and <code>✓ EQUAL</code> .
1:20–2:20	Blind fault	Test bench, <code>Start blind test</code> . A visitor selects a case. Point at Trends → CUSUM crossing 5 before any rule trips (the early warning), then the WATCH and WARNING alerts and the diagnosis card. Press <code>Reveal the card</code> : <code>✓ CORRECT</code> .
2:20–3:20	Misfire	Inject Cylinder misfire on C3. Turn <code>Sound on</code> : the engine stumbles. On Vibration a 0.5× line appears and the polar dot lands in the C3 sector (–45°). The torque command rises by 4/3 . The card names "Cylinder 3 misfire".
3:20–3:50	Sensor fault	Repair, then Sensor fault: COOLANT, SPIKE. The card says "Coolant sensor fault": the reading jumped 25 °C in 0.1 s (about 250 K/s) but the energy balance allows 0.81 K/s. The cooling system is not blamed.
3:50–4:30	Prognosis and report	Oil-pump wear with onset <code>PROGRESSIVE</code> at 10× for a minute: the health panel shows a remaining-life band. Raise the Load slider: wear speeds up and the RUL shrinks. Report → <code>Print / save PDF</code> .
4:30–5:00	Hardware path	<code>SAY</code> Every source emits the same telemetry object. An ESP32 with cheap sensors (DS18B20, MAX31855, a pressure transducer, an ADXL345) or a USB OBD-II adapter only needs one adapter file; the analytics do not change. Next step: a heat-balance run on a real engine rig.

Questions visitors ask, and short answers

QUESTION	ANSWER
Isn't it just reading the fault you injected?	No. The analytics only receive telemetry; the fault lives in a module they cannot import (a lint rule enforces it). The blind test proves it.
How accurate is the model?	A reduced-order model with demo calibration, checked against hand calculations on the Validation page. Absolute accuracy needs calibration on a real engine.

Why not deep learning?	There is no real failure data. A network trained on our simulator would just learn our own equations back. Our statistics have a known false-alarm rate; machine learning becomes useful once real data is logged.
What is the false-alarm rate?	D ² uses the χ^2 99 % limit, so about 1 healthy sample in 100 crosses it. An alarm needs 3 s of persistence. In our tests a healthy engine never raised a held alarm.
Why does an inline-4 shake at twice crank speed?	The first-order piston forces cancel between the pairs of cylinders; the second-order forces add. That is 2,517 N peak at 3,000 rpm.
How do you know which cylinder misfired?	From the phase of the half-order crank-speed component: +45°, +135°, -45°, -135° for cylinders 1, 2, 3, 4.
Why exactly 4/3?	The governor holds the speed, so mean torque must still match the load. With one cylinder out, three pulses carry four pulses' worth.
Sensor spike or real overheating?	Physics: the engine cannot heat faster than about 0.8 K/s, and a real rise would also warm the oil through the water jacket.
Is the RUL real?	It is a trend estimate: a straight-line fit with a ± 2 standard-error band, shown only when the slope is significant, and conditional on the current load.

If something goes wrong on stage

SYMPTOM	RECOVERY
The 3D view stutters	Keep going: the numbers are unaffected. Or reload with <code>?no3d</code> .
A step is taking too long	Test bench → Time-warp <code>60x</code> , or press <code>Restart</code> on the scenario strip.
A wrong fault is still active	Test bench → <code>Repair</code> . Alerts clear after their hold time.
Lost track of the state	Reload the page. It restarts from a cold engine with the same seed.

Replicate it tomorrow: checklist

Do the first block tonight. Do the second block on the day. Nothing here needs the internet except the first install.

Tonight

- 1 **Node is installed.** Open a new terminal: `node --version` shows 20.19 or higher.
- 2 **The project is on the presentation laptop** (not just your own). Double-click `start-ignisense.bat` once **with internet**, keep the window open until the browser opens.
- 3 **Turn the Wi-Fi off and run it again.** It must still open. That proves it works offline.
- 4 **Run the Full walkthrough once** (Test bench → Scenario → Full walkthrough → Run scenario) and watch it to the end.
- 5 **Check the 3D frame rate** on *that* laptop (top of the 3D view). "Reduced effects" is fine if it stays smooth; if it does not, plan to use `?no3d`. **GATE**
- 6 **Phone?** If you will show it on a phone, check it at 375–400 px on that phone first. The desktop layout is approved. **GATE**
- 7 **Hardware?** Keep ESP32 / OBD-II as a roadmap slide unless that exact hardware has passed a full rehearsal. **GATE**
- 8 **Test the sound** on the laptop's speakers. Adjust the laptop volume, not the app.
- 9 **Open Validation** and confirm every row is green.
- 10 **Save a spare report:** run a fault, Report → `Print / save PDF`, keep the PDF on the desktop.
- 11 **Practise the blind test** once with a colleague selecting the case.

On the day

- 1 Close other heavy programs and plug the laptop in (battery saver slows the 3D view).
- 2 Start IgniSense 10 minutes early. Set the browser to full screen (`F11`) and zoom so the gauges are readable from a distance.
- 3 Decide who drives and who talks. The person talking should know chapters 5 and 6 well enough to answer "how does it know?".
- 4 Keep this guide and `docs/whiteboard.md` open for the derivations: $\omega = 314 \text{ rad/s}$, $4/3$, 0.8 K/s , $F_2 = 2,517 \text{ N}$.
- 5 Use **Safe** mode (the tour) with visitors who only stop for a minute, **Live** mode when someone wants to pick a fault.
- 6 Between groups press `Repair` then reload if anything looks odd.

THE 30-SECOND VERSION

SAY "This is a digital twin of a petrol engine. The real engine hides a fault; the twin only sees the sensors. The difference between what the sensors read and what a healthy engine would read is the evidence. From that we name the fault, find which cylinder, estimate how long is left, and we can show the equation behind every number. Choose a fault and watch the monitor find it."

Build it yourself: phases and project map

IgniSense was built one phase at a time. Each phase ends with tests that must pass, so this is also the order to rebuild it in.

PHASE	WHAT WAS BUILT	WHERE, AND HOW IT WAS CHECKED
0	Scaffold and guard rails	Vite, React, TypeScript strict, Tailwind, Vitest, ESLint. The rule that stops the analytics importing the plant. Seeded RNG.
1	Slow physics	<code>src/physics/</code> : pure functions for friction, energy, cooling, oil, electrical. Golden numbers reproduce within 2 %.
2	Plant, sensors, Twin, worker	<code>plant/</code> , <code>twin/</code> , <code>sources/simSource.ts</code> , <code>worker/</code> . Healthy residuals are pure noise; same seed, same telemetry.
3	Faults, residuals, diagnosis, health, alerts	<code>analytics/</code> . Each fault gives the right top diagnosis; healthy 20 min gives no alarm.
4	Live dashboard	<code>ui/pages/LiveTwin</code> , <code>Trends</code> , zustand store, uPlot charts.
5	3D engine	<code>three/</code> : procedural I4 driven by the slider-crank equation.
6	Oil-pump scenario	<code>worker/scenarios.ts</code> ; three runs identical.
7	Crank angle, vibration, misfire	<code>physics/crankTorque</code> , <code>vibration</code> ; <code>analytics/fft</code> , <code>spectral</code> . Every misfire golden row reproduces.
8	Calculation view, blind test	<code>physics/registry.ts</code> , <code>ui/math/</code> , <code>worker/blind.ts</code> . Substituted numbers equal the gauges.
9	Advanced analytics	Sensor sanity, D^2 , CUSUM, RUL, extra faults, stress-dependent wear.
10	Validation, report, sound, hot city	<code>ui/pages/Validation</code> , <code>Report</code> ; <code>ui/audio/</code> .
11	Demo hardening	Full walkthrough, start scripts, offline check, <code>docs/demo-script.md</code> .
12	Open-questions review	<code>docs/open-questions-review.md</code> : decisions that need the physics owner.
13	Real data sources	<code>sources/csv</code> , <code>replaySource</code> , <code>serialSource</code> ; ESP32, OBD-II, phone accelerometer.

Folder map

FOLDER	HOLDS
<code>src/physics/</code>	Pure equations, each registered with LaTeX, inputs and a compute function (that is what the calculation view reads). No state.
<code>src/engine/profile.ts</code>	Every engine parameter with units in the name (<code>bore_m</code> , <code>uaFan_Wperk</code>). Change a value here, then run <code>npm test</code> .
<code>src/plant/</code>	The "real" engine state, hidden faults, sensor model, crank and accelerometer signals.
<code>src/twin/</code>	The healthy reference. Driven only by measured speed, load, ambient.
<code>src/analytics/</code>	Residuals, sanity checks, statistics (D^2 , CUSUM), FFT, diagnosis, health, alerts, remaining life. May not import plant or sources.
<code>src/sources/</code>	Anything that produces telemetry: simulator, CSV replay, ESP32 / OBD-II.
<code>src/worker/</code>	The simulation loop (<code>simLoop.ts</code>), scenarios, blind deck, the worker wrapper, and the message types.
<code>src/three/</code>	The 3D scene.

<code>src/ui/</code>	Pages, design-system components, math drawer, audio, the test bench.
<code>docs/</code>	The research review and build plan (source of truth), calibration log, open questions, demo script, whiteboard derivations.

Where to change common things

I WANT TO...	CHANGE
Change an engine parameter (mass, efficiency, radiator size...)	<code>src/engine/profile.ts</code> , then log it in <code>docs/calibration.md</code>
Change alert thresholds, weights, misfire ramps	<code>src/analytics/config.ts</code>
Change how the engine sounds (level, brightness, growl)	<code>tone()</code> in <code>src/ui/audio/engineSound.ts</code>
Change the team or product name	<code>src/brand.ts</code> (used everywhere)
Add a scenario	<code>src/worker/scenarios.ts</code>
Add a new data source (a real sensor board)	One new file in <code>src/sources/</code> that outputs the same <code>Telemetry</code> object. The analytics do not change.

RULES THAT MUST NOT BE BROKEN

The analytics must never import fault or plant state (the lint rule stops it). Physics stays pure, with no imports from plant, twin, analytics or the UI. All randomness goes through the seeded generator. Sensor dropout is `null`, never 0 or NaN. Every new physics function gets a test against a hand-calculated value.

Key numbers cheat-sheet

The values the tests check and the demo quotes. All are demo calibration.

Basics at 3,000 rpm

Angular speed ω	314.16 rad/s
Crank frequency	50 Hz
Firing frequency	100 Hz
Brake power at 80 N·m	25.1 kW
BMEP at 80 N·m	5.03 bar

Oil pressure (bar)

Cold idle, 20 °C	2.54
Hot idle, 100 °C	1.39
Hot, 3,000 rpm	3.23
Pump 0.4, hot idle	0.56
Bearing wear 0.6, 3,000	1.70

Vibration F_2 (N)

800 rpm (26.7 Hz)	179
3,000 rpm (100 Hz)	2,517
6,000 rpm (200 Hz)	10,068

Misfire, 3,000 rpm / 80 N·m

Healthy ripple	11 rpm p-p
One cylinder out	61 rpm p-p
0.5× amplitude	21.0 rpm
Torque command	110 → 146 N·m (×4/3)
Phases C1, C2, C3, C4	+45, +135, -45, -135°

Statistics

$\chi^2(5)$, 99 %	15.09
CUSUM κ , h	0.5, 5
Max coolant heating	≈ 0.8 K/s
Health example (draft §22.1)	69.4

Steady-state coolant and energy flow

POINT	BRAKE KW	FUEL KW	COOLANT KW	FUEL L/H	STEADY COOLANT
Idle 800 rpm	0	7.5	2.6	0.85	82 °C (warm-up)
3,000 rpm, 80 N·m	25.1	110.8	31.9	12.5	93 °C
4,000 rpm, 190 N·m	79.6	252.3	55.5	28.6	98 °C (fan cycling)
6,000 rpm, full load	103.6	370.4	81.5	41.9	98 °C (fan cycling)

Faults: cooling health 0.5 at full load reaches 132 °C; cooling health 0.25 at 3,000 rpm / 80 N·m reaches 110 °C.

Limitations to state honestly

Saying these yourself is a strength. It shows you know what the model is and is not.

- The telemetry is **synthetic**. It is not a substitute for engine-test data, and every coefficient is demo calibration.
- The thermal model is **lumped**: one coolant temperature and one oil temperature, with no spatial gradients.
- The lubrication model is an **empirical pressure law**, not a bearing-film (Reynolds) solver.
- The crank model ignores compression torque and cycle-to-cycle combustion scatter. The governor is modelled at its settled state, not as a transient controller.
- Vibration amplitudes are calibrated to look sensible, **not measured**. Thresholds are engine-profile calibration values, never ISO limits.
- A slow drift in the oil-pressure or voltage sensor looks exactly like pump or alternator wear, because there is no physical cross-check for those channels. (Coolant has one: the oil must follow a real coolant rise.)
- The remaining life is conditional on the assumed wear law and on the load continuing. Stop-and-go driving makes the trend noisy, so none is shown until it is significant.
- The evidence scores are weighted sums, **not calibrated probabilities**. That is why the app never says "confidence %".
- The overall health is a weighted average, so it can look mild beside one critical subsystem. The alert level carries the critical override.
- ESP32 and OBD-II are not yet tested on real hardware. 60 fps depends on the laptop's graphics card.

Glossary

TERM	MEANING
Digital twin	A live model of a physical thing, run alongside it and compared with it.
Plant	In IgniSense, the simulated "real" engine, the only place hidden faults exist.
Twin	The healthy reference model. It predicts what each sensor should read.
Telemetry	The one common data format every source produces (rpm, load, temperatures, pressure, voltage, and so on).
Residual (r, z)	Measured minus expected; z is that gap in units of the sensor's noise.
Twin marker	The white tick or pointer showing the Twin's expected value on a gauge.
Health factor	A 0-to-1 multiplier on part of the physics. 1 = healthy.
Severity	How bad an injected fault is, 0 to 1.
Evidence score	The weighted 0-to-1 support for a fault. Not a probability.
CUSUM	Cumulative sum of small residuals; catches a slow persistent drift early.
D ² (Mahalanobis)	A single number for how unusual the whole set of residuals is compared with healthy running.
χ^2 limit	The threshold D ² exceeds only about 1 time in 100 for a healthy engine (15.09 for 5 signals).
RUL	Remaining useful life: how long until health reaches the failure level, at the current trend and load.

Hysteresis	Requiring a condition to persist (or clear) for some time, so alerts do not flicker.
Order (0.5×, 1×, 2×)	A multiple of crank speed. Order tracking keeps a fault at the same order at any rpm.
F₂	The second-order reciprocating force that makes an inline-4 shake at twice crank speed.
Misfire	A cylinder that fails to fire, removing one power pulse every two revolutions.
Time-warp	Running the simulated clock faster than real time (1×, 10×, 60×).
Demo calibration	A parameter chosen to give sensible behaviour, not measured on a real engine.

IgniSense — Smart Engine Health Diagnostic · Team Revora · Physics-informed engine digital twin. For deeper detail see [docs/research-review-and-build-plan.md](#), [docs/calibration.md](#), [docs/demo-script.md](#) and [docs/whiteboard.md](#).